MASTER'S PROJECT / GPGPU

Break the GPU

Think Parallel. Make It Scale. Push the Limits.

The challenge

Choose a computational problem and push its GPU implementation as far as you can. Design a working solution, establish a performance baseline, identify bottlenecks, and improve it through controlled experiments. **Your reasoning and evidence matter more than achieving the highest absolute speed.**

Choose your problem

The topic is open: image processing, rendering, simulation, scientific computing, data analysis, cybersecurity, cryptography, or another GPU-suitable application. Examples include particle simulation, ray tracing, parallel pattern matching, hashing, sorting, and reductions. These are suggestions, not prescribed subjects.

Technologies

WebGPU / WGSL is recommended for its portability and potential for browser-based demonstrations. CUDA, Vulkan Compute, OpenCL, and other GPU technologies are also permitted. You may use the development tools of your choice; be prepared to explain your decisions.

What we expect

1. **Problem and parallel strategy** — Explain the task and why GPU parallelism is appropriate.
2. **Correct implementation** — Validate outputs against a reference or suitable tests.
3. **Baseline and measurements** — Record a meaningful starting point and the experimental setup.
4. **Optimization** — Try alternative approaches and measure the impact of each change.
5. **Analysis** — Explain bottlenecks, trade-offs, unsuccessful attempts, and remaining limitations.

Deliverables

- **Source code** with concise instructions and any requirements needed to understand or reproduce the work.
- **Short demonstration video** showing the result, how it works, and the main optimizations. A live browser demo is encouraged for WebGPU projects, but does not replace the video.
- **Concise experimental summary** with baseline, performance comparisons, hardware / software configuration, and interpretation of the results.

Experimental Approach

GPU performance depends on hardware, workload, and implementation. Explore different problem sizes, parallelization strategies, and memory layouts. Compare implementations under consistent conditions, and support your conclusions with measurements rather than assumptions.

Assessment

Projects will be evaluated on GPU understanding, correctness, experimental rigor, quality of optimization, and clarity of communication. **A well-explained optimization that fails to improve performance is still a useful result.**

Guiding principle Don't just make it run on the GPU. Measure it, understand it, and make it better.